Ders 3 Genişletilmiş Notu: Algoritma Analizi, Asimptotik Notasyon ve Insertion Sort

Ders 3 Genişletilmiş Notu: Algoritma Analizi, Asimptotik Notasyon ve Insertion Sort

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Bir Algoritmanın Sağlaması Gereken Şartlar
- 2. Verimlilik: Zaman ve Yer
- 3. Çalışma Süresini Etkileyen Faktörler
- 4. Adım Sayarak Karmaşıklık Hesabı
- 5. Temel İşlem ve Sadeleştirme
- 6. Asimptotik Analiz ve Asimptotik Notasyon
- 7. Best, Worst ve Average Case
- 8. Doğrusal Arama ve Analizi
- 9. Hangi Durum Önemli?
- 10. Insertion Sort ve Analizi
- 11. Karmaşıklık Sınıfları
- 12. Kullanışlı Toplam Formülleri
- Örnek Sorular
- Dersin Toparlaması

1. Bir Algoritmanın Sağlaması Gereken Şartlar

Hoca bu dersin konusunun, yazılan bir algoritmanın analizinin nasıl yapılacağı olduğunu söyledi. Bu konu üçüncü sınıftaki Algoritma Analizi dersinde haftalarca ayrıntılı işleniyor. Veri Yapıları ve Algoritmalar dersinde ise yazdığımız bir algoritmayı nasıl analiz edeceğimiz ana hatlarıyla anlatılıyor.

Derse, önceki haftada konuşulan, bir algoritmanın sağlaması gereken şartlar hatırlatılarak başlandı:

- **Doğru (correct) olmak:** Algoritma verilen farklı girişler için her zaman doğru sonucu vermelidir. Ayrıca sonlu sayıda adımdan oluşmalı, sonsuz döngüye girmemeli, belli bir zamanda sonlanmalıdır. Bir algoritmada ilk sağlanması gereken şart budur.
- **Basit (simple) olmak:** Hoca bunu bir önceki haftada anlatılan Occam'ın usturası (Occam's razor) ile bağladı. Aynı problemi çözmenin ideal yolu, onu basit yoldan çözmektir. Okunduğunda rahat anlaşılabilen, karmaşık olmayan bir algoritma tasarlamak önemlidir.
- **Genel (general) olmak:** Algoritma yalnız belli değerler, belli sayılar için değil, problemin genel hâli için çalışmalıdır. Tasarım buna göre yapılmalıdır.
- **Kolay gerçeklenebilir (easy to implement) olmak:** Tasarımda çok kolay görünen bir şey gerçeklemeye geçince karmaşık, içinden çıkılmaz bir hâl alıyorsa algoritma iyi tasarlanmamış, karmaşık çözümler düşünülmüş demektir. O zaman "bu problemi daha basit bir şekilde çözebilir miyim?" diye indirgemenin yolları aranmalıdır.
- **Verimli (efficient) olmak:** İlk dört şart bir algoritmada mutlaka bulunmalıdır. Bunlar sağlandıktan sonra üzerinde durulan şey, algoritmanın ne kadar verimli olduğudur.

Basitlik konusunda hoca, önceki haftada söylediği bir tavsiyeyi tekrarladı: Doğrudan kod yazmaya geçmek yerine önce algoritmayı sembolik olarak tasarlamak gerekir. Bunun en kolay yollarından biri akış diyagramı (flowchart) çizmek, diğeri yarı kod (pseudo code) yazmaktır. Sembolik tasarımdan sonra bir hata analizi yapılırsa hatalar görülür ve algoritmanın üzerinden geçilerek gerekli düzenlemeler yapılır. Doğrudan kod yazıldığında ise hata düzeltme genellikle "yama" şeklinde olur. Bu da algoritmanın olmaması gereken bir şekilde büyümesine ve basitliğinin bozulmasına yol açar.

> **Ek bilgi (derste anlatılmadı):** Occam'ın usturası, büyük olasılıkla 14. yüzyıl düşünürü Ockhamlı William'a atfedilen bir ilkedir. Kabaca "bir olguyu açıklayan seçenekler arasında gereksiz varsayım içermeyen en basiti tercih edilmelidir" der. Algoritma tasarımında karşılığı, aynı işi yapan çözümler içinde gereksiz adım ve karmaşıklık içermeyenin seçilmesidir.

2. Verimlilik: Zaman ve Yer

Hoca verimliliği ölçmek için algoritmanın iki kaynağı nasıl kullandığına bakıldığını hatırlattı: zaman (time) ve yer (space).

2.1 Büyüme Hızlarının Karşılaştırılması

Bir algoritmanın çalışma süresi giriş sayısına doğrudan bağlıdır. 10 elemanlı bir diziyi sıralamak ile 100 bin elemanlı bir diziyi sıralamak farklı sürer. Hoca, karmaşıklığın çalışma süresini nasıl etkilediğini göstermek için farklı n değerleri için çalışma sürelerini veren bir tablo gösterdi. Tablodaki değerler, her işlemin yaklaşık 1 ns sürdüğü bir bilgisayar için verilmiştir:

Karmaşıklık	Derste verilen örnek	$n = 10$	$n = 10^8$
$\log n$	İkili arama (binary search)	0.003 µs	0.027 µs
n	Dizinin en büyük elemanını bulma, dizide eleman arama	0.01 µs	0.1 s
$n \log n$	Merge sort	0.033 µs	2.66 s
n^2	Selection sort	0.1 µs	115.7 gün
2^n		1 µs	$n = 100$ için 4×10^{13} yıl
$n!$		3.63 ms	

Hoca değerleri önce milisaniye olarak okudu, sonra "bunlar mikrosaniye" diye düzeltti. $n = 10$ iken bütün algoritmalar mikrosaniye mertebesinde çalışıyor; $n!$ karmaşıklıklı algoritma bile 3-4 milisaniyede bitiyor ve bunda "çok zararlı bir şey yok". Ama n 100 milyona çıkınca tablo tamamen değişiyor: Doğrusal bir algoritma 0.1 saniyeye, merge sort 2.66 saniyeye çıkıyor, selection sort ise yaklaşık 115.7 gün sürüyor. $n = 100$ için bile 2^n karmaşıklıklı bir algoritma 4×10^{13} yıl sürüyor.

Hoca burada gerçek dünyaya dikkat çekti: Gerçek problemlerde 10 veya 20 elemanla uğraşmayız. Bir bankanın müşterileri dendiğinde milyonlarca müşteri vardır; bir üniversitenin öğrencileri dendiğinde 50 bin, 60 bin öğrenci vardır. 50 bin öğrenci için çalışacak bir algoritmanın yavaş olmaması, zamanı iyi kullanması gerekir.

Sıralama örneği üzerinden devam etti: $n \log n$ karmaşıklıklı merge sort ve quicksort, n^2 karmaşıklıklı bubble sort, selection sort ve insertion sort vardır. Bunların hepsi aynı işi yapıyorsa ve büyük bir veri (bin, on bin, yüz bin eleman) sıralanıyorsa mantıken merge sort veya quicksort seçilir. Bunlar arasında neye göre seçim yapılacağı ileriki yıllarda uzun uzun görülecek. Buradaki ana fikir şudur: Aynı işi yapan iki algoritma varsa seçim, zamanı iyi kullanandan yana olmalıdır. Bu dönem Veri Yapıları ve Algoritmalar'da başlayıp üçüncü sınıfın ilk döneminde Algoritma Analizi dersinde devam eden hedef, algoritmaları daha verimli tasarlamaya yardımcı olacak bilgileri vermektir.

Ders sonrası not: Altyazıda $n = 10$ için $\log n$ satırı "0.03" geçiyor; hesapla $\lg 10 \approx 3.32$ işlem × 1 ns ≈ 0.0033 µs, yani 0.003 µs çıkar. $n!$ için hoca "3.66 ms" dedi; $10! = 3\,628\,800$ ns $= 3.63$ ms'dir. n^2 satırı altyazıda "%10.5.7 gün" diye bozuk geçiyor; hoca derste sonra "115 gün" ve "115.7 gün" dedi. Hesap: 10^{16} ns $= 10^7$ s ≈ 115.74 gün. 2^{100} ns ise yaklaşık 4.02×10^{13} yıl eder; tablodaki değer doğrudur.

2.2 Yerin Verimli Kullanılması

Hoca öğrencilere birinci sınıfta selection sort ve bubble sort görüp görmediklerini sordu; öğrenciler gördüklerini söyledi. Bu sıralama algoritmalarında hep "yedek dizi kullanılmayacak" denir ve işlem aynı dizi üzerinde yapılır. Sebebi yeri verimli kullanmaktır. Eleman sayısı çok fazla olduğunda yalnız zaman değil, yer sorunu da vardır. Bir milyon elemanlık bir diziyi yedek bir diziye almak, yeri dikkatli kullanmamak demektir.

2.3 Neden Yeniden Önemli Oldu?

Hoca kısa bir tarihçe verdi. İlk bilgisayarlarda zaman ve yer kullanımı sıkıntılıydı: Kullanılabilen bellek çok kısıtlı, işlemciler yavaştı. Sonra, özellikle oyunların da etkisiyle donanımlar hızlandı. Son yıllarda ise iki sebepten ötürü zamanı ve yeri iyi kullanmak yeniden önemli hâle geldi:

- **Gömülü sistemler (embedded systems):** Bilgisayar gibi davranan ama bilgisayar olmayan sistemlerdir. Günlük hayatta kullandığımız akıllı telefonlar bir gömülü sistemdir. Bunlarda hızlı çalışan algoritmalar için zamanı ve yeri iyi kullanmak gerekir.
- **Derin öğrenme (deep learning) ile çalışan yapay zekâ algoritmaları:** Milyonlarca veri üzerinde çalışılır ve çok karmaşık matematiksel işlemler içeren algoritmalar kullanılır. Bir modelin eğitimi çok iyi donanımlarda bile bir-iki ay sürebilir.

3. Çalışma Süresini Etkileyen Faktörler

Bir algoritmanın zamanı iyi kullanıp kullanmadığını nasıl değerlendireceğiz? İlk akla gelen yol, algoritmayı çalıştırıp süresini ölçmektir: kaç milisaniye, kaç saniye sürdüğüne bakmak. Hoca bunun iyi bir çözüm olmadığını söyledi. A bilgisayarının donanımı ile B bilgisayarınınki farklıysa aynı algoritma farklı hızlarda çalışır. "Algoritmam 3 milisaniyede çalışıyor" dendiğinde derdi anlatmak çok zordur; hangi şartlarda çalıştığını iyi tarif etmek gerekir. Karşı taraf da kendi şartlarında ne kadar süreceğini ayrıca hesaplamak zorunda kalır.

Üstelik çalışma süresini etkileyen tek faktör donanım değildir:

- **Donanım:** İşlemcinin hızı, bellek miktarı gibi doğrudan donanımla ilgili özellikler.
- **Derleyici (compiler):** Aynı C kodu, kullanılan derleyiciye göre makine koduna farklı verimlilikte dönüşür. Kod üretimi (code generation) sırasında ne kadar verimli kod üretildiği çalışma hızını etkiler.
- **Programlama dili:** Aynı program C'de, Java'da farklı hızlarda çalışır. Assembly dilinde yazıldığında makine diline çok yaklaşıldığı için çok daha hızlı çalışır.
- **Veri:** Hocaya göre bizim için önemli olan faktör budur.
  - Verinin sayısı: 10 elemanlı dizi ile 100.000 elemanlı dizi için aynı algoritmanın çalışma süreleri aynı değildir.
  - Başlangıç sıralaması (initial ordering): Elemanlar baştan sıralıysa bir dizide eleman aramanın hızı farklı olur. Sıralıya yakın bir diziyi sıralamak ile tamamen rastgele karışık bir diziyi sıralamak farklı sürer.

İlk üç faktör sisteme bağlı etkilerdir. Tasarlanan algoritmanın kendisi ise sistemden bağımsızdır; kullanılan makineden, programlama dilinden ve derleyiciden bağımsız olarak çalışma süresini etkiler. Bu yüzden bir algoritmanın çalışma süresi hakkında çıkarım yapmak için onu çalıştırıp süresine bakmak doğru değildir. Doğrusu algoritmayı tek başına değerlendirmektir; böylece sisteme bağlı etkilerden bağımsız olarak algoritmanın her ortamda ne durumda olduğu söylenebilir.

Bunun için önce algoritmanın en iyi durumda, en kötü durumda ve ortalama durumda nasıl çalıştığı belirlenmelidir. Hoca burada kısa bir örnek verdi: Sırasız `3, 5, 7, 9, 8` dizisinde bir

eleman aranıyor olsun. En iyi durumda (best case) aranan 3'tür, bakar bakmaz bir adımda bulunur. En kötü durumda (worst case) aranan 8'dir veya dizide olmayan 1'dir; o zaman dizinin sonuna kadar bakmak gerekir. Ortalama durum (average case) ise "ortalama almak" değildir: Eleman olasılıksal olarak birinci, ikinci, üçüncü… sırada olsaydı kaç adımda bulunurdu, bunların beklenen değerinin belirlenmesidir. Hoca bunu dersin ilerleyen kısmında bir kez göstereceğini söyledi.

Karmaşıklık (complexity), girişteki eleman sayısı n iken algoritmanın n'e bağlı olarak kaç adım süreceğinin belirlenmesidir.

4. Adım Sayarak Karmaşıklık Hesabı

Karmaşıklığı hesaplamanın en temel yolu, her işlemin kaç kez yapıldığını tek tek saymaktır. Hoca tahtaya, bir dizideki sıfırları sayan küçük bir algoritma parçası yazdı:

```
int count = 0, i, n;
for (i = 0; i < n; i++)
{
    if (a[i] == 0)
        count++;
}
```

Sonra işlemleri türlerine göre tek tek saydı ve bazılarını öğrencilere sordu:

İşlem	Sıklık
Değişken tanımlama (variable declaration)	3
Atama (assignment)	2
`i < n` karşılaştırması (less than compare)	$n+1$
`a[i] == 0` karşılaştırması (equal compare)	n
Dizi erişimi (array access)	n
Artırma (increment)	n ile $2n$ arası

`i < n` neden $n+1$ kez? Bir öğrenci önce "n" dedi, hoca "bir daha dikkat" deyince "$n+1$" cevabı geldi. i sıfırdan başlar ve son olarak $(n-1)$'inci elemanla iş yapar; n elemanlı bir dizinin indisleri 0'dan $n-1$'e kadardır. $i = 0, \dots, n-1$ için n kontrol yapılır. Döngüden çıkmak için $i = n$ olduğunun da anlaşılması gerekir; bu da $(n+1)$'inci karşılaştırmadır.

`a[i] == 0` neden n kez? Bu kontrol döngünün içindedir ve döngünün içine n kez girilir. Hoca iki kontrolü özellikle ayırdı: Döngü koşulu $n+1$ kez, döngü gövdesindeki koşul n kez kontrol edilir. Dizi erişimi de aynı şekilde n kez yapılır.

Artırma kaç kez? Burada iki artırma vardır: `i++` ve `count++`. `i++` her zaman n kez yapılır. `count++` ise `a[i]` sıfır oldukça yapılır: Dizide hiç sıfır yoksa hiç yapılmaz, dizinin tamamı sıfırsa n kez yapılır, arada birkaç sıfır varsa o kadar yapılır. Toplam artırma sayısı n ile $2n$ arasındadır. Öğrencilerden "$2n$ mi?" ve sonra "n ile $2n$ arasında mı?" cevapları geldi; hoca ikincisini onayladı.

Bu sırada bir öğrenci Ayrık Matematik dersindeki en kötü durum hesabını hatırlatarak "en kötü ihtimal n" dedi. Hoca bu sorunun best/worst case ile ilgili olmadığını, yalnız `count++` işinin kaç kez yapıldığını sorduğunu söyledi. Ayrıca tahtada `count+++` diye üç artı yazdığını fark edip öğrencilerin kendisini uyarmadığını belirtti; doğrusu `count++`'dır.

En kötü durumda toplam şöyle olur (ara adım eklendi):

$$3 + 2 + (n+1) + n + n + 2n = 5n + 6$$

Hiç sıfır olmayan bir dizide ise artırma n kez yapılır ve toplam $4n+6$ olur. Bu sayımlar, adımları sayan bir C programıyla $n = 10$ için doğrulandı: Tamamı sıfır olan dizide 56 ($= 5 \cdot 10 + 6$), hiç sıfır olmayan dizide 46 ($= 4 \cdot 10 + 6$) adım çıktı.

5. Temel İşlem ve Sadeleştirme

5.1 Küçük Farkların Önemi Var mı?

Hoca şu durumu düşündürdü: Aynı basit algoritmayı yazan iki öğrencinin birinde 2-3 komut fazla, diğerinde eksik olabilir. Birinin algoritması tek tek sayıldığında $4n+6$, diğerininki $4n+25$ çıkabilir; bir başkası fazladan bir döngü yazmışsa $5n+12$ çıkabilir. Bu farkların önemi var mı?

Öğrencilerin "yok sanırım" cevabını hoca tablo üzerinden gerekçelendirdi. n karmaşıklıklı bir algoritma 100 milyon eleman için 0.1 saniye sürüyor; $n \log n$ olunca 2.66 saniyeye çıkıyor. Yani n'e bağlı bir çarpan eklenince büyük bir etki oluşur. Ama n yerine $n+10$ adım olsa, 100 milyon eleman için 0.1 saniyeye yalnız yaklaşık 0.01 mikrosaniye eklenir. Bu kadar küçük bir şeyin önemi yoktur. Tabloda $n = 10$ iken bile 3-5 tane sabit işlemin etkisi, 2^n ve $n!$ algoritmalarında bile çok küçüktür. Sonuç: Komut komut saymak mantıklı değildir; daha genel bir yönteme gitmek gerekir.

5.2 Temel İşlem

Genelleme için önce **temel işlem (basic operation)** belirlenir. Temel işlem, n'e bağlı olarak tekrarlanan, en çok yapılan iştir. "A'ya 1 atadım, B'ye 5 atadım" gibi işler değil, n kez, n^2 kez, yani n'e bağlı değişken sayıda yapılan işler önemlidir. Çünkü n on bin, yüz bin, bir milyon olabilir; algoritma bir milyon eleman için çalıştırıldığında bile sabit işler yine bir kez yapılır. Bu yüzden bir algoritmanın karmaşıklığını belirlerken ilk yapılacak şey temel işlemi belirlemektir.

Bir algoritmada önce n kez dönen bir döngü, ardından n^2 kez dönen başka bir döngü varsa karmaşıklık n^2 dönen döngüye göre hesaplanır; n'li döngünün çok büyük bir katkısı yoktur.

5.3 Sadeleştirme Adımları

Hoca karmaşıklığı $4n^2 + 2n + 9$ olan bir algoritma üzerinden üç adımı gösterdi:

1. **Sabitlerden kurtul:** $4n^2 + 2n + 9 \rightarrow 4n^2 + 2n$. 100 milyon eleman için çalışan bir algoritmada 9 tane değişken atamasının hiçbir etkisi yoktur.
2. **Katsayılardan kurtul:** Algoritmanın $4n^2$ olması ile $5n^2$ olması arasında ciddi bir fark yoktur. Etkileyen şey n^2 veya n olmasıdır.
3. **En önemli bileşeni al:** $n^2 + n$ olan bir algoritmada n'in bir etkisi yoktur. Sonuç n^2 olur.

Bu terimlerin etkisizliği n **çok büyükken,** $n \rightarrow \infty$ **iken** geçerlidir; $n = 10$ iken durum farklıdır. Tablodan: $n = 10^8$ iken n^2 karmaşıklıklı bir algoritma 115.7 gün sürüyorsa n^2+n karmaşıklıklı bir algoritma 115.7 gün artı 0.1 saniye sürer. $n = 100$ iken 2^n karmaşıklıklı bir algoritma 4×10^{13} yıl sürüyorsa $2^n + n^2$ olanın da farkı yoktur.

Bu sadeleştirme işi kolaylaştırır. Sıfır sayma örneğinde $4n+6$ ya da $5n+6$ için artık doğrudan n denebilir. Tek yapılması gereken, tek tek saymak yerine döngüye bakmaktır: Döngü n kez dönüyorsa karmaşıklık n'dir.

Python ile $\frac{4n^2+2n+9}{4n^2}$ oranına bakıldığında $n = 10$ için 1.07, $n = 1000$ için 1.0005, $n = 10^6$ için 1.0000005 çıkar; yani küçük terimlerin etkisi n büyüdükçe kaybolur.

5.4 Tablodaki Sayılar Gerçek mi?

Soru: Tablodaki sayılar gerçek ölçüm mü, yoksa büyümeyi göstermek için verilmiş örnek değerler mi?

Cevap: Hoca küçük değerlerin aynı bilgisayarda gerçekten çalıştırılarak elde edildiğini söyledi: Örneğin aynı makinede 10 eleman için binary search, en büyük elemanı bulma gibi algoritmalar çalıştırılmış. 115.7 gün veya 4×10^{13} yıl gibi değerler ise matematiksel olarak hesaplanmıştır. 10 veya 50 eleman için sürenin nasıl olduğu bilinince 100 milyon eleman için ne kadar süreceği hesaplanabilir; "yoksa ömür yetmez", 2^n'i çalıştırarak bulmak mümkün değildir.

6. Asimptotik Analiz ve Asimptotik Notasyon

Yazılan bir algoritmanın donanımdan, derleyiciden ve programlama dilinden bağımsız olarak analiz edilmesine **asimptotik analiz (asymptotic analysis)** denir. Asimptotik analizde n'e bağlı olarak karmaşıklık hesaplanır. Akılda n'in sonsuza giden büyük değerleri olduğu için katsayılar ve sabit terimler elenir, en yüksek anlamlı terim alınır. Örneğin $n \log n$ yanında bir de $+n$ varsa, $+n$'in değeri yoktur.

Aslında algoritmanın gerçek karmaşıklığı n^2 değil, örneğin $4n^2 + 2n + 5$'tir. $n \to \infty$ varsayımıyla bunu n^2 olarak söyleyebilmemizi sağlayan gösterime **asimptotik notasyon (asymptotic notation)** denir. Üç tanesi vardır:

- **Big O:** Asimptotik üst sınırı (upper bound) gösterir. $O(\cdot)$ biçiminde yazılır.
- **Big Theta (Θ):** Sıkı sınırı (tight bound) gösterir.
- **Big Omega (Ω):** Alt sınırı (lower bound) gösterir.

  Hocanın vurgusu: Theta ve Omega ileriki yıl Algoritma Analizi dersinde anlatılacak. Bu dönem karmaşıklıkları göstermek için yalnız Big O kullanılacak.

Asimptotik üst sınırın anlamı şudur: Karmaşıklığı $4n^2 + 5n$ olan bir algoritma $O(n^2)$ olarak gösterilebilir. Aslında $O(n^3)$ veya $O(n^4)$ olarak göstermek de mümkündür, çünkü üst sınır, $4n^2+5n$'den yüksek olan herhangi bir şey olabilir. Bunlar teknik olarak doğrudur ama mantıklı değildir: Kimse n^2 karmaşıklıkla yazdığı bir algoritma için daha karmaşık bir şey söylemek istemez. Bu yüzden böyle bir algoritma $O(n^2)$ olarak gösterilir.

> **Ek bilgi (derste anlatılmadı):** Big O'nun biçimsel tanımı şöyledir: $f(n) = O(g(n))$ demek, öyle pozitif bir c sabiti ve bir n_0 vardır ki her $n \geq n_0$ için $f(n) \leq c \cdot g(n)$ olur demektir. Örneğin $4n^2 + 5n \leq 9n^2$ eşitsizliği her $n \geq 1$ için sağlanır, bu yüzden $4n^2 + 5n = O(n^2)$'dir. Aynı tanım $g(n) = n^3$ için de sağlandığından $O(n^3)$ yazmak da yanlış değildir; hocanın "doğru ama mantıklı değil" dediği durum budur.

6.1 İki Boyutlu Döngülerde Dikkat

Hoca burada bir nüansa dikkat çekti. Matrislerle çalışan, iç içe iki döngüden oluşan bir algoritmanız olsun: m satır sayısını, n sütun sayısını göstersin ve karmaşıklık $m \cdot n$ çıksın. Buna $O(n)$ demek doğru mudur? Ne de olsa "en büyük terim dışındakilere bakmıyoruz" demiştik.

Bir öğrenci "m de değişebildiği için doğru olmaz" dedi; hoca bunu "güzel" diye onayladı ve ayrıntılandırdı. Karar algoritmanın ne yaptığına bakılarak verilir:

- m'nin sınırlı, küçük değerlerde kalacağından eminseniz (örneğin en fazla 100) ve m, n ile doğru orantılı değişmiyorsa $O(n)$ diyebilirsiniz.
- m de n gibi büyüyorsa (örneğin n bin olduğunda m 700, n 100.000 olduğunda m 50.000 oluyorsa) ya $O(n^2)$ ya da kullanım yerine göre $O(m \cdot n)$ demelisiniz. İkisi de doğrudur.

7. Best, Worst ve Average Case

Karmaşıklığın nasıl gösterileceği anlaşıldıktan sonra hoca, ne olunca best case, ne olunca worst case, ne olunca average case oluştuğuna geçti.

7.1 Dizinin En Büyük Elemanı

Hoca 6 elemanlı bir dizi çizdi ve en büyük elemanı bulmada best case ile worst case'in değişip değişmediğini sordu. Bir öğrenci "değişmez" dedi; hoca onayladı. Best case'de de worst case'de de n adımda bulunur. Başka şans yoktur: En büyüğü bulmak için dizinin sonuna kadar bakmak gerekir.

7.2 Sırasız Dizide Eleman Arama

Bazı algoritmalarda ise durum farklıdır. `2, 1, 4, 7, 9, 4` dizisinde bir x sayısı aranıyor olsun. $x = 2$ ise bir adımda bulunur. $x = 9$ ise 1, 2, 3, 4, 5 adımda bulunur. Adım sayısı sayının nerede durduğuna bağlıdır.

Buradan genel bir kural çıkar: Best case ve worst case meselesi **adım sayısı belli olmayan algoritmalarda** anlam kazanır. Bunlar genelde `while` ile yazılan, bir iş bitince (bir koşul sağlanınca) çıkılan algoritmalardır. Adım sayısı belli değilse en iyi durum, en kötü durum ve ortalama durum birbirinden farklı sonuçlar verir.

8. Doğrusal Arama ve Analizi

Hoca sırasız bir dizide verilen bir elemanın yerini bulan algoritmayı, döngüyü ve her şeyi bir arada görüp hesabı rahat yapmak için doğrudan kod üzerinden gösterdi. Bunun birinci sınıf birinci dönemde görülmüş temel bir algoritma olduğunu söyledi:

```
int search(int a[], int n, int x)
{
    int i = 0;
    while (i < n && a[i] != x)
        i++;
    if (i < n)
        return i;
    else
        return -1;
}
```

Algoritma x sayısını dizide arar. Dizinin sonuna gelene kadar veya bulana kadar i artırılarak ilerlenir. Döngüden iki sebeple çıkılmış olabilir: Ya $i = n$ olmuştur (bulunamadı), ya da `a[i] == x` olmuştur (bulundu). Çağıran taraf -1 alırsa elemanın bulunamadığını anlar.

8.1 Hocanın İzlemesi

Dizi `3, 5, 1, 2, 7`, $n = 5$.

$x = 1$ **için:**

i	`a[i]`	`i < n`	`a[i] != x`	Sonuç
0	3	evet	$3 \neq 1$, evet	i artırılır
1	5	evet	$5 \neq 1$, evet	i artırılır
2	1	evet	$1 \neq 1$, hayır	döngüden çıkılır

Dışarıda $2 < 5$ olduğu için 2 döndürülür. Gerçekten 1 sayısı 2 numaralı gözdedir.

$x = 4$ **için:** $i = 0$ (3), $i = 1$ (5), $i = 2$ (1), $i = 3$ (2), $i = 4$ (7) için hepsi 4'ten farklıdır. $i = 5$ olur, `5 < 5` yanlıştır ve döngüden çıkılır. Dışarıda da `5 < 5` yanlış olduğu için -1 döndürülür.

Bu iki izleme `gcc -Wall` ile derlenen programla doğrulandı: $x = 1$ için 2 (3 karşılaştırma), $x = 4$ için -1 (5 karşılaştırma) döndü.

8.2 Temel İşlem

Hoca temel işlemin döngüdeki karşılaştırma (`a[i] != x`) olduğunu gösterdi. Temel işlem, n'e bağlı olarak en çok yapılan iştir ve hesap onun üzerinden yapılır.

8.3 Worst Case

Aranan eleman dizide yoksa veya son gözdeyse n karşılaştırma yapılır. Tek tek sayarak gidilir; n'inci göze kadar gelinir, sonra ya bulunamamış ya da son gözde bulunmuş olur. Worst case verimliliği:

$$C_{worst}(n) = n \Rightarrow O(n)$$

8.4 Best Case

Aranan eleman ilk gözdeyse (örnekte 3 aranıyorsa) ilk karşılaştırmada `a[0] != x` yanlış çıkar ve döngüden çıkılır. Bir karşılaştırma yapılır:

$$C_{best}(n) = 1 \Rightarrow O(1)$$

8.5 Average Case

> **Hocanın vurgusu:** Average case bu dönem yalnız bir kez, aklınızda yer etsin diye gösteriliyor. Dönem boyunca hep worst case üzerinden ilerlenecek. Average case Algoritma Analizi dersinde ele alınacak.

Average case, rastgele giriş için beklenen adım sayısıdır (expected steps for random input). Aranan eleman ikinci gözde, beşinci gözde, yedinci gözde gibi rastgele bir yerde olabilir; average case bunların olasılıkla ağırlıklı ortalamasıdır.

Kurulum:

- p, aranan elemanın dizide bulunma olasılığı olsun. Olasılık her zaman 0 ile 1 arasında bir değerdir.
- Hoca, elemanın i'inci sırada olma olasılığını sordu. Rastgele durumda elemanın birinci sırada olması ile ikinci sırada olması eşit olasılıklıdır; öğrenciler de "eşittir" dedi. Özel bir durum, bir sıra yoktur. n eleman varsa ve bulma olasılığı p ise i'inci pozisyonda bulma olasılığı $\frac{p}{n}$'dir.
- Eleman i'inci pozisyondaysa yapılan karşılaştırma sayısı i'dir. Örneğin eleman dördüncü sıradaysa dört karşılaştırma yapılır.
- Eleman dizide yoksa (olasılık $1 - p$) n karşılaştırma yapılır.

Toplam iki kısma ayrılır: elemanın dizide olduğu durum (başarılı arama, successful search) ve olmadığı durum (başarısız arama, unsuccessful search).

$$C_{avg}(n) = \underbrace{\left[1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + \cdots + n \cdot \frac{p}{n}\right]}_{\text{successful search}} + \underbrace{n(1-p)}_{\text{unsuccessful search}}$$

$\frac{p}{n}$ parantezine alınırsa:

$$C_{avg}(n) = \frac{p}{n}(1 + 2 + \cdots + n) + n(1-p) = \frac{p}{n} \cdot \frac{n(n+1)}{2} + n(1-p) = \frac{p(n+1)}{2} + n(1-p)$$

İki özel durum:

- $p = 1$ **(başarılı arama):** İkinci terim sıfır olur; $C_{avg}(n) = \frac{n+1}{2}$.
- $p = 0$ **(eleman dizide yok):** $C_{avg}(n) = n$ olur. Bu zaten worst case'dir; eleman dizide yoksa dizinin sonuna kadar gidilir.

Hoca average case'i $\frac{n+1}{2}$ olarak aldı. Sabitler atılınca:

$$C_{avg}(n) = \frac{n+1}{2} \Rightarrow O(n)$$

Bu formül python ile kesirli aritmetikle $n = 1, 5, 7, 10$ ve $p = 0, \frac{1}{2}, 1$ için toplam açılarak doğrulandı. Ayrıca 6 elemanlı dizinin bütün permütasyonlarında her elemanın aranması sayılınca başarılı aramanın ortalaması $\frac{7}{2} = \frac{n+1}{2}$ çıktı.

8.6 $C(n)$'deki C Ne Demek?

Soru: Best, worst ve average case'i gösterirken yazdığımız $C(n)$'deki C ne anlama geliyor?

Cevap: Hoca C'yi karşılaştırmanın (compare) veya karmaşıklığın (complexity) kısaltması olarak kullandığını söyledi. Time'ın T'si ile $T(n)$ de yazılabilir; fark etmez. Önemli olan, bir algoritmanın karmaşıklığı sorulduğunda sonucu Big O ile, örneğin $O(n)$ veya $O(1)$ olarak vermektir.

9. Hangi Durum Önemli?

Hoca best, average ve worst case'ten hangisinin önemli olduğunu sordu; bir öğrenci "worst" dedi.

- **Best case'in hiçbir önemi yoktur.** Çok şansa bağlı bir durumdur; istenen sonucun ilk seferde elde edilmesi beklenen bir şey değildir.
- **Normalde düşünülmesi gereken worst case'dir.** Bir işi yaparken en kötü durumda ne olacağı düşünülür.
- **Kaynaklar yeterli değilse plan average case'e göre yapılır.**

Hoca average case planlamasının sonuçlarına örnekler verdi:

- **GSM operatörleri:** Telefon şirketleri sistemlerini ortalama trafiğe, yani normal şartlarda aynı anda ortalama kaç kişinin arayacağına göre kurar; bu daha ucuzdur. Depremde görüldüğü gibi milyonlarca kişi aynı anda birbirini aramaya kalktığında, yani worst case oluştuğunda sistem çöker. Sebep, aslında sağlayamayacakları değil, average case'e göre plan yapmalarıdır.
- **ÖSYM:** Üniversite sınav sonuçları açıklandığı anda ÖSYM'nin sitesi çöker. Site, aynı anda ortalama kaç kişinin bağlanacağına göre planlanmıştır; aynı anda çok fazla istek gelince kaldıramaz.
- **Bayram sabahı:** Özellikle eskiden, bayram sabahları herkes aynı anda birbirini aradığı için yavaşlık olurdu.

Hoca burada bir soru daha sordu: Bu sistemlerde average case mi kullanılmalıydı, worst case mi? Öğrenci "worst case" dedi. Hoca "gönül worst case'i istiyor" ama sorgulanması gerekenin maliyet olduğunu söyledi: Servis sağlayıcılar planlamayı worst case'e göre yapsa maddi olarak ne kadar fark ederdi, yoksa bunu yalnız eski alışkanlıklardan mı yapıyorlar? Maliyetler artık çok ucuzladığı için hoca bunun cevabını merak ettiğini belirtti. Sonuç olarak ihtiyaca ve şartlara göre worst case'e mi, average case'e mi göre plan yapılacağına karar vermek gerekir.

9.1 Quicksort Örneği

Hoca son olarak quicksort'u örnek verdi. Quicksort'un average case karmaşıklığı $O(n \log n)$, worst case karmaşıklığı ise $O(n^2)$'dir. Buna rağmen en çok kullanılan sıralama algoritması quicksort'tur; hoca Java'nın da içinde quicksort kullandığını söyledi. Merge sort ve heap sort gibi başka $O(n \log n)$ algoritmalar olmasına rağmen quicksort'un tercih edilmesinin sebebi, normalde average case'in oluşacağının düşünülmesidir.

> **Ders sonrası not:** Hoca Java'nın sıralamada quicksort kullandığını söyledi. Bu altyazıdan doğrulanamaz; büyük olasılıkla Java ilkel tipli dizilerde (int, double gibi) Dual-Pivot Quicksort, nesne dizilerinde ise merge sort ile insertion sort'u birleştiren TimSort kullanır. Küçük parçalarda insertion sort'a geçilmesi hocanın söylediğiyle uyumludur.

10. Insertion Sort ve Analizi

Hoca best, worst ve average case'i birbirinden farklı olan algoritmalara bir örnek olarak insertion sort'u (araya yerleştirme sıralaması) hatırlattı. Bazı grupların bunu Bilgisayar Bilimlerine Giriş dersinde gördüğünü, bazılarının görmediğini söyledi.

10.1 Algoritmanın İzlemesi

Dizi `3, 5, 2, 1, 4, 7, 8` küçükten büyüğe sıralanacak. Karşılaştırmaya ikinci elemandan başlanır. Hocanın tahtadaki anlatımı:

- **5'in yerleştirilmesi:** 5, 3 ile karşılaştırılır. 5, 3'ten büyük olduğu için yerinde kalır.
- **2'nin yerleştirilmesi:** 2, 5 ile karşılaştırılır; yer değiştirir. Sonra 2, 3 ile karşılaştırılır; yer değiştirir. Dizi `2, 3, 5, 1, 4, 7, 8` olur.
- **1'in yerleştirilmesi:** 1 sırayla 5, 3 ve 2 ile karşılaştırılır ve her seferinde öne geçer. Dizi `1, 2, 3, 5, 4, 7, 8` olur.
- **4'ün yerleştirilmesi:** 4, 5 ile karşılaştırılır ve öne geçer. Sonra 3 ile karşılaştırılır; 4, 3'ten büyüktür, burada durulur. Dizi `1, 2, 3, 4, 5, 7, 8` olur. 2 ve 1 ile karşılaştırmaya gerek yoktur: Baş taraf her zaman sıralı olduğu için 4, 3'ten büyükse 2'den ve 1'den de büyüktür.
- **7'nin yerleştirilmesi:** 7, 5 ile karşılaştırılır. 7, 5'ten büyük olduğu için 1, 2, 3, 4'e bakmaya hiç gerek yoktur.
- **8'in yerleştirilmesi:** 8, 7'den büyük olduğu için baş tarafla karşılaştırmaya gerek yoktur.

Sonuç `1, 2, 3, 4, 5, 7, 8`'dir.

Adım	Yerleştirilen	Karşılaştırma	Dizi
Başlangıç			3, 5, 2, 1, 4, 7, 8
1	5	1 (3)	3, 5, 2, 1, 4, 7, 8
2	2	2 (5, 3)	2, 3, 5, 1, 4, 7, 8
3	1	3 (5, 3, 2)	1, 2, 3, 5, 4, 7, 8
4	4	2 (5, 3)	1, 2, 3, 4, 5, 7, 8
5	7	1 (5)	1, 2, 3, 4, 5, 7, 8
6	8	1 (7)	1, 2, 3, 4, 5, 7, 8

Toplam 10 karşılaştırma yapılır. Ara diziler ve karşılaştırma sayısı, aşağıdaki kodun `gcc -Wall` ile derlenip çalıştırılmasıyla doğrulandı.

> **Hocanın vurgusu:** Insertion sort'un $O(n^2)$ karmaşıklıklı diğer algoritmalara göre avantajı budur: Baş taraf her zaman sıralı olduğu için yerleştirilen eleman solundakinden büyükse daha geriye bakmaya gerek kalmaz. Selection sort'ta ve bubble sort'ta karşılaştırmaya hep devam etmek zorundasınız.

10.2 Verimlilik İpuçları

Hoca algoritmayı yazmadan önce iki küçük iyileştirmeye dikkat çekti:

- **Yerleştirilecek elemanı geçici bir değişkene almak:** Sürekli `A[i]` ile `A[j]`'yi karşılaştırmak, her seferinde dizinin i'inci ve j'inci adreslerine erişmeyi gerektirir ve yavaştır. Bir turda örneğin 4 sürekli dizinin geri kalan elemanlarıyla karşılaştırılacaksa, önce geçici bir değişkene (`tmp`) alınıp sonra hep `tmp` ile karşılaştırmak daha verimlidir.
- **Yer değiştirmek yerine kaydırmak:** Dizi `3, 5, 2, 1, 4, 7, 8` hâlindeyken 2 yerleştirilecekse "2 ile 5'i değiştir, sonra 2 ile 3'ü değiştir" demek yerine 5'i bir sağa, 3'ü bir sağa kaydırıp 2'yi doğru yere koymak daha doğrudur. Yer değiştirmede her seferinde iki eleman birden oynar. 2 zaten geçici değişkende olduğu için onu oradan oraya oynatmaya gerek yoktur; zamanı gelince doğru yere yerleştirilir.

Hoca bunun, algoritmayı tasarlarken çok ufak iyileştirmelerle daha verimli hâle getirmenin mümkün olduğunu gösterdiğini söyledi. Bu fikre C dilinden bir örnek de ekledi: `i = i + 1` yerine `i++` yazmak. Döngüye 10 milyon kez giriliyorsa `i++` demek daha verimlidir, çünkü `i++` düşük seviyede, Assembly'de görülen tek bir increment komutuna karşılık gelir; `i = i + 1` ise bir add komutuna karşılık gelir.

> **Ders sonrası not:** Hoca `i++`'nın `i = i + 1`'den daha hızlı olduğunu söyledi. Bu makinedeki gcc ile iki biçimi içeren aynı döngü `-O0` (optimizasyonsuz) ve `-O2` ile Assembly'ye derlendi; iki biçim de her iki seviyede aynı makine kodunu üretti, yalnız etiket adları farklıydı. Yani en azından gcc ile bir hız farkı oluşmaz. Hocanın dediği büyük olasılıkla, derleyicinin iki ifadeyi farklı çevirdiği eski veya basit derleyiciler için geçerlidir.

10.3 Akış Diyagramı ve Kod

Hoca algoritmayı tahtada akış diyagramıyla tasarladı. Mantığı şöyle anlattı:

- İşe birinci gözden (indis 1) başlanır ve $(n-1)$'inci göze kadar bütün elemanlara tek tek bakılır. Bu yüzden i, 1'den $n-1$'e kadar birer birer artırılır.
- Sürekli `A` ile karşılaştırma yapmamak için `A[i]` geçici değişkene alınır: `tmp = A[i]`.
- Eleman kendisinden öncekilerle karşılaştırılacağı için j, i'den başlatılır.
- Her seferinde `tmp`, `A[j-1]` ile karşılaştırılır. `tmp` daha küçükse `A[j-1]`, `A[j]` gözüne yerleştirilir (sağa kaydırılır) ve j bir azaltılır. Hoca akış diyagramında `j--` yerine `j = j - 1` yazdığını, çünkü dile özgü komutları akışta göstermemenin faydalı olduğunu söyledi. Döngü koşulundaki iki şart arasına da "ve" (and) yazdı.
- İç döngüden çıkınca `A[j] = tmp` yapılır.

Akış diyagramının C karşılığı ve hocanın örnek dizisiyle tam bir program:

```c
#include <stdio.h>

void insertion_sort(int A[], int n)
{
    int i, j, tmp;
    for (i = 1; i <= n - 1; i++) {
        tmp = A[i];
        j = i;
        while (j > 0 && tmp < A[j - 1]) {
            A[j] = A[j - 1];
            j = j - 1;
        }
        A[j] = tmp;
    }
}

int main(void)
{
    int A[] = {3, 5, 2, 1, 4, 7, 8};
    int n = 7, k;

    insertion_sort(A, n);
    for (k = 0; k < n; k++)
        printf("%d ", A[k]);
    printf("\n");
    return 0;
}
```

Çıktı: `1 2 3 4 5 7 8`.

$i = 2$ için hocanın izlemesi: Dizi `3, 5, 2, 1, 4, 7, 8`, 3 ve 5 yerleşmiş, sıra 2'de.

Adım	j	Karşılaştırma	İşlem	Dizi
Başlangıç	2		`tmp = 2`	3, 5, 2, 1, 4, 7, 8
1	2	$2 < A[1] = 5$, evet	`A[2] = 5`, $j = 1$	3, 5, 5, 1, 4, 7, 8
2	1	$2 < A[0] = 3$, evet	`A[1] = 3`, $j = 0$	3, 3, 5, 1, 4, 7, 8
3	0	$j > 0$ yanlış	döngüden çıkılır, `A[0] = 2`	2, 3, 5, 1, 4, 7, 8

Hoca "3, 5, 5, 1, 4 vaziyetindeyiz" diyerek ara durumu da gösterdi. Dış döngü bittiğinde `A` dizisi yazdırılırsa sıralı bir dizi elde edilir.

> **Ders sonrası not:** Hoca izlemenin sonunda "2, 3, 5, 4 oldu" dedi. $i = 2$ turundan sonra dizi `2, 3, 5, 1, 4, 7, 8`'dir; 1 henüz yerleştirilmemiştir.

10.4 Karmaşıklık Analizi

Algoritmada iç içe iki döngü vardır: dıştaki `for` ve içteki `while`. Temel işlem, en çok tekrar eden iş olan iç döngüdeki karşılaştırmadır.

Worst case: Dizi büyükten küçüğe sıralıdır: $A[0] > A[1] > \cdots > A[n-1]$. Her eleman baş taraftaki bütün elemanlardan küçüktür ve en başa kadar gitmesi gerekir.

- $i = 1$ için 1 karşılaştırma,
- $i = 2$ için 2 karşılaştırma,
- ...
- $i = n - 1$ için $n - 1$ karşılaştırma.

$$C_{worst}(n) = 1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2} \Rightarrow O(n^2)$$

Best case: Dizi zaten küçükten büyüğe sıralıdır: $A[0] < A[1] < \cdots < A[n-1]$. Her turda yalnız bir karşılaştırma yapılır ve eleman yerinde kalır.

$$C_{best}(n) = \underbrace{1 + 1 + \cdots + 1}_{n-1} = n - 1 \Rightarrow O(n)$$

Hoca best case'i `3, 5, 7, 9` sıralı dizisiyle gösterdi: 5, 3 ile karşılaştırılır, büyüktür, durur; 7, 5 ile karşılaştırılır, durur; 9, 7 ile karşılaştırılır, durur. Her adımda bir karşılaştırma yeter.

Derlenen programla $n = 10$ için ters sıralı dizide 45 ($= \frac{10 \cdot 9}{2}$), sıralı dizide 9 ($= n - 1$) karşılaştırma sayıldı; `3, 5, 7, 9` için 3 karşılaştırma çıktı. Python ile $n = 5, 10, 100$ için de aynı formüller doğrulandı.

> **Hocanın vurgusu:** Insertion sort'un best case'i $O(n)$, worst case'i $O(n^2)$'dir.

10.5 Insertion Sort Nerede Kullanılır?

- **Küçük diziler:** Hoca Java'nın küçük diziler için kendi içinde insertion sort kullandığını söyledi. Quicksort diziyi parçalara bölerek sıralar; küçük bir parça üzerinde çalışırken orada da insertion sort'tan yararlanılabilir.
- **Sıralıya yakın diziler:** Yerinde olmayan yalnız birkaç eleman varsa insertion sort'un karmaşıklığı n'e yakın olur. Genelde zaten sıralı olan bir diziye birkaç yeni eleman gelip en sona eklendiğinde, onları doğru yere yerleştirmek için insertion sort kullanılır.

10.6 Hangi Sıralama Algoritması Nerede?

Soru: Bu sıralama algoritmalarından hangisini nerede kullanmamız gerektiğini nereden anlayabiliriz?

Cevap: Hoca bunun Algoritma Analizi dersinde ayrıntılı analizlerle anlatıldığını söyleyip kısa bir cevap verdi:

- Bubble sort ve selection sort çok kullanılan sıralama algoritmaları değildir.
- Çok kullanılanlar $O(n \log n)$ olmaları sebebiyle quicksort, merge sort ve heap sort'tur.
- $O(n^2)$ olanlar bubble sort, insertion sort, selection sort gibi algoritmalardır; radix sort gibi başkaları da ileride görülecek.
- Quicksort'un average case'i $O(n \log n)$, worst case'i $O(n^2)$'dir. Worst case dizinin ters sıralı olduğu durumdur; normalde rastgele dizilerde hep $n \log n$ çalışır, bu yüzden en çok quicksort tercih edilir.
- Bundan emin değilseniz merge sort tercih edilebilir; merge sort her zaman $O(n \log n)$ çalışır. Ama içinde yedek dizi kullanır; bu bir dezavantajdır.
- Selection sort'un best, worst ve average case'i fark etmez, hep $O(n^2)$'dir. Bu yüzden tercih edilmemelidir. $O(n^2)$ algoritmalar içinde en çok tercih edilen insertion sort'tur.

- 50, 100 elemanlı bir dizi için $O(n \log n)$ karmaşıklıklı bir algoritmayla problemi çözmek gerekmez. Bu algoritmalar özyinelemeli (recursive) çalışır ve sürekli alt program çağırmanın kendi yükü vardır. Bu yüzden küçük dizilerde insertion sort tercih edilir.

Karar verirken bunun gibi birçok parametrenin birden dikkate alınması gerekir.

> **Ders sonrası not:** Hoca quicksort'un worst case'inin ters sıralı dizi olduğunu söyledi. Bu, pivotun ilk (veya son) eleman seçildiği basit quicksort için geçerlidir ve o sürümde zaten sıralı dizi de worst case'dir. İlk elemanı pivot alan bir quicksort python ile denendi: 500 elemanlı sıralı dizide de, ters sıralı dizide de $\frac{500 \cdot 499}{2} = 124\,750$ karşılaştırma yapıldı; rastgele karıştırılmış dizide ise 4705 karşılaştırma yapıldı ($n \lg n \approx 4483$ mertebesi).

11. Karmaşıklık Sınıfları

Hoca dersin sonunda algoritma türlerinden karmaşıklık durumlarına göre kısaca bahsetti:

Notasyon	Ad	Derste verilen örnekler
$O(1)$	Sabit zamanlı (constant time)	İki sayıdan küçüğünü bulma, bir yılın artık yıl olup olmadığını bulma, hashing
$O(\log n)$	Logaritmik zamanlı (logarithmic time)	Binary search; her adımda problemin ikiye bölündüğü döngüler
$O(n)$	Doğrusal zamanlı (linear time)	Dizinin en büyük elemanını bulma; içeride n kez iş yapılan algoritmalar
$O(n \log n)$	Linearitmik (linearithmic)	Quicksort, merge sort
$O(n^2)$	Karesel zamanlı (quadratic time)	Selection sort, bubble sort; iç içe iki döngü olan algoritmalar
$O(2^n)$	Üstel zamanlı (exponential time)	

Sabit zaman: Örneğin 2000 yılı artık yıl mıdır sorusu birinci sınıfın başında görülen problemlerdendir, ama gerçekte de bazen böyle işler yapmak gerekir. Hoca ileriki yıl Algoritma Analizi'nde görülecek hashing algoritmasının da sabit karmaşıklıklı olduğunu söyledi. $O(1)$, "yalnız bir karşılaştırma yapılıyor" demek değildir. 10, 20, hatta 50 karşılaştırma yapılabilir; önemli olan bunun n'e bağlı olmaması, n ile büyümemesidir. $O(n)$'de ise $n = 100$ iken 100, $n = 100.000$ iken 100.000 karşılaştırma yapılır.

Logaritmik zaman: Döngüde $n = n/2$ gibi her adımda ikiye bölünerek gidiliyorsa logaritmik bir azalma olur. 1024 eleman varsa bir adım sonra yarısı, bir adım sonra onun da yarısı kalır ve 10 adımda 1'e inilir. Algoritmada döngü ikiye bölüne bölüne gidiyorsa 2 tabanında logaritmik bir algoritma olduğu anlaşılır.

> **Ders sonrası not:** Hoca 1024 elemanın yarılanmasını "500, 250" diye yuvarlak söyledi. Doğru dizi $1024, 512, 256, 128, 64, 32, 16, 8, 4, 2, 1$'dir ve gerçekten 10 adımda 1'e inilir ($\log_2 1024 = 10$); python ile doğrulandı.
>
> **Ders sonrası not:** Hoca $n \log n$ algoritmaları için "linear big time" benzeri bir ad kullandı (altyazıda böyle geçiyor). Yerleşik İngilizce terim "linearithmic" veya "$n \log n$ time"dır.
>
> **Hocanın vurgusu:** Bu sınıfları ezberlemeniz asla gerekmiyor. Bilmeniz gereken yalnız şu: Sabitse $O(1)$, ikiye bölüne bölüne gidiyorsa $O(\log n)$. Diğerleri, derste n'i nasıl çıkardıysak, döngülerden çıkarılarak bulunur.

Hoca algoritma analizinin genel mantığını şöyle özetledi: Kendimizce olabildiğince verimli bir algoritma tasarlarız ama gerçekten verimli olup olmadığından emin değilizdir. Bunu anlamak için karmaşıklığını hesaplarız. Karmaşıklık $n \to \infty$ için hesaplandığından sabitler, katsayılar

ve küçük terimler elenir ve sonuç asimptotik notasyonla, bu dönem yalnız Big O ile, en büyük terim korunarak gösterilir.

12. Kullanışlı Toplam Formülleri

Hoca selection sort'u herkes bildiği için yeniden anlatmadığını söyleyip karmaşıklık hesabında kullanılabilecek basit toplam bağıntılarını hatırlattı:

- Birlerin toplamı: $\sum_{i=1}^{n} 1 = n$
- Alt sınırı l, üst sınırı u olan birler toplamı ($l \leq u$ şartıyla): $\sum_{i=l}^{u} 1 = u - l + 1$
- İlk n doğal sayının toplamı: $\sum_{i=1}^{n} i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2} \approx \frac{n^2}{2}$
- Kareler toplamı: $\sum_{i=1}^{n} i^2 = 1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6} \approx \frac{n^3}{3}$
- Sabit çarpan dışarı çıkar: $\sum_{i=l}^{u} c \cdot a_i = c \sum_{i=l}^{u} a_i$
- Logaritmalar toplamı: $\sum_{i=1}^{n} \log i \approx n \log n$

İlk dört formül python ile $n = 1, 7, 50$ için ve $l = 3, u = 9$ için doğrulandı. "Yaklaşık" ifadeleri asimptotiktir: $\frac{n(n+1)(2n+1)/6}{n^3/3}$ oranı $n = 10$ için 1.155, $n = 1000$ için 1.0015 olur ve 1'e yaklaşır.

> **Ek bilgi (derste anlatılmadı):** $\sum_{i=1}^{n} \log i = \log(n!)$'dir ve Stirling yaklaşımından $\log(n!) = n \log n - n \log e + O(\log n)$ çıkar. Yani bu toplam $n \log n$'den bir miktar küçüktür ama aynı büyüme hızındadır; python ile $\frac{\sum \lg i}{n \lg n}$ oranı $n = 10$ için 0.66, $n = 10^6$ için 0.93 bulundu ve oran yavaşça 1'e yaklaşır. Hocanın "$\approx n \log n$" ifadesi bu asimptotik anlamdadır.

Örnek Sorular

Bu derste ekrandan alınmış soru karesi yoktur. Aşağıdaki soruların hepsi hocanın derste sözlü olarak sorduğu sorulardır.

1. (derste sözlü, 00:29:43) Sırasız, n elemanlı bir dizide (örneğin `3, 5, 7, 9, 8`) bir elemanın yerini arıyoruz. En iyi durumda elemanı kaç adımda buluruz? En kötü durumda?

> **Çözüm:** En iyi durumda aranan eleman ilk gözdedir (3), bir adımda bulunur. En kötü durumda aranan eleman son gözdedir (8) veya dizide yoktur (1); dizinin sonuna kadar bakılır, n adım gerekir. Derlenen programda 3 için 1, 8 ve 1 için 5 karşılaştırma sayıldı.

2. (derste sözlü, 00:34:23) Sıfır sayma döngüsünde (`for (i = 0; i < n; i++)`) `i < n` karşılaştırması kaç kez yapılır?

> **Çözüm:** $n+1$ kez. $i = 0, \ldots, n-1$ için n kontrol yapılır; $(n+1)$'inci kontrolde $i = n$ olduğu anlaşılır ve döngüden çıkılır.

3. (derste sözlü, 00:34:51) Aynı döngüde `a[i] == 0` karşılaştırması kaç kez yapılır?

> **Çözüm:** n kez, çünkü bu kontrol döngünün içindedir ve döngünün içine n kez girilir.

4. (derste sözlü, 00:37:05) Aynı algoritmada artırma (`i++` ve `count++`) işi kaç kez yapılır?

> **Çözüm:** n ile $2n$ arası. `i++` her zaman n kez, `count++` dizideki sıfır sayısı kadar (0 ile n arası) yapılır. En kötü durumda toplam adım sayısı $5n + 6$ olur.

5. (derste sözlü, 00:40:43) Aynı işi yapan algoritmalardan biri $4n + 6$, diğeri $4n + 25$, bir başkası $5n + 12$ adım çıkarsa bu farkların önemi var mı?

> **Çözüm:** Yoktur. n çok büyükken sabit terimler ve katsayılar çalışma süresini anlamlı ölçüde değiştirmez; hepsi $O(n)$'dir.

6. (derste sözlü, 01:06:08) İç içe iki döngü m satır ve n sütun üzerinde çalışıyor, karmaşıklık $m \cdot n$ çıkıyor. Buna $O(n)$ demek doğru mudur?

> **Çözüm:** Genelde doğru değildir, çünkü m de değişebilir. m sınırlı ve küçük kalıyorsa (örneğin en fazla 100) $O(n)$ denebilir. m de n gibi büyüyorsa $O(n^2)$ veya $O(m \cdot n)$ denmelidir.

7. (derste sözlü, 01:08:53) Bir dizinin en büyük elemanını bulmada best case ile worst case değişir mi?

> **Çözüm:** Değişmez. En büyüğü bulmak için dizinin sonuna kadar bakmak zorunludur; her iki durumda da n adım gerekir.

8. (derste sözlü, 01:19:14) Aranan elemanın dizide bulunma olasılığı p ise, rastgele durumda elemanın i'inci sırada olma olasılığı nedir?

> **Çözüm:** Rastgele durumda bütün pozisyonlar eşit olasılıklıdır, bu yüzden olasılık $\frac{p}{n}$'dir.

9. (derste sözlü, 01:20:20) Eleman i'inci pozisyondaysa doğrusal aramada kaç karşılaştırma yapılır?

> **Çözüm:** i karşılaştırma. Örneğin eleman dördüncü sıradaysa dört karşılaştırma yapılır. Bunlardan ortalama durum $C_{avg}(n) = \frac{p(n+1)}{2} + n(1-p)$ çıkar; $p = 1$ için $\frac{n+1}{2}$, yani $O(n)$'dir.

10. (derste sözlü, 01:27:56) Best case, average case ve worst case'ten hangisi önemlidir?

> **Çözüm:** Best case şansa bağlı olduğu için önemsizdir. Normalde worst case esas alınır. Kaynaklar yetersizse veya maliyet önemliyse average case'e göre plan yapılabilir (GSM operatörleri, ÖSYM sitesi örnekleri).

11. (derste sözlü, 01:31:02) Telefon operatörleri ve ÖSYM gibi sistemlerde average case mi kullanılmalıydı, worst case mi?

> **Çözüm:** Öğrenci "worst case" dedi. Hoca kesin bir cevap vermedi: Worst case'e göre planlama tercih edilir, ama bunun maliyet farkının sorgulanması gerektiğini söyledi. Karar, ihtiyaca, şartlara ve maliyete göre verilir.

Dersin Toparlaması

Bu derste bir algoritmanın verimliliğinin, çalıştırıp süresini ölçerek değil, donanım, derleyici ve programlama dilinden bağımsız olarak, giriş boyutu n'e göre adım sayısıyla değerlendirildiği görüldü. Komut komut sayma yöntemi sıfır sayma örneğinde gösterildi ($5n + 6$), sonra bunun yerine n'e bağlı en çok tekrarlanan işin, yani temel işlemin sayılmasının yeterli olduğu anlatıldı. $n \to \infty$ için sabitler, katsayılar ve küçük terimler atılarak sonuç asimptotik notasyonla verilir; bu dönem yalnız Big O kullanılacak. Adım sayısı girişe göre değişen algoritmalarda best, worst ve average case ayrımı ortaya çıkar. Doğrusal aramada bunlar $O(1)$, $O(n)$ ve $\frac{n+1}{2} \Rightarrow O(n)$ olarak hesaplandı. Dönem boyunca worst case esas alınacak.

Insertion sort, baş tarafın hep sıralı olmasından yararlanan, geçici değişken ve kaydırma ile verimli yazılan bir $O(n^2)$ algoritma olarak incelendi: Worst case'i (ters sıralı dizi) $\frac{n(n-1)}{2} \Rightarrow O(n^2)$, best case'i (sıralı dizi) $n - 1 \Rightarrow O(n)$'dir. Bu yüzden küçük ve sıralıya yakın dizilerde tercih edilir. Son olarak karmaşıklık sınıfları ($O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$) ve karmaşıklık hesabında kullanılan toplam formülleri hatırlatıldı. Hoca bu konuların Theta, Omega ve average case analizleriyle birlikte ayrıntılı hâlinin üçüncü sınıftaki Algoritma Analizi dersinde işleneceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.